

面向 Qwen2.5-Coder-32B 的存量前端系统 AI Coding 上下文治理 PRD

1. 文档信息

项目	内容
文档名称	面向 Qwen2.5-Coder-32B 的存量前端系统 AI Coding 上下文治理 PRD
文档版本	V1.0
适用范围	存量前端系统、旧前端项目、待接入 AI Coding 的业务系统
目标模型	Qwen2.5-Coder-32B
主要使用对象	前端开发人员、系统负责人、前端技术小组、项目管理人员
核心目标	为旧系统生成模型可理解、可检索、可约束的上下文资料，支撑 Qwen2.5-Coder-32B 稳定生成和修改代码

2. 项目背景

随着 AI Coding 能力逐步引入研发流程，前端开发场景中出现了利用大模型辅助新增页面、修改字段、接入接口、修复报错、补充文档等需求。但在存量前端系统中，普遍存在以下问题：

1. 项目文档缺失，系统结构依赖核心开发人员经验传承。
2. 旧系统技术栈复杂，存在 Vue2、Vue3、React、Umi、Webpack、Vite 等多种形态。
3. 公共组件、业务组件、接口封装、权限规则、字典规则分散在代码中，缺乏统一说明。
4. 页面与接口调用关系不清晰，开发人员和模型都难以快速理解业务链路。
5. 旧系统构建压力大，全量 build 和全量质量检查成本高。
6. 中小规模代码模型在长上下文理解、多文件复杂改造和项目全局推理上存在一定不稳定性。
7. 如果直接让模型基于自然语言需求生成代码，容易出现编造组件、编造接口、乱 import、修改范围过大、代码风格不一致等问题。

因此，需要建设一套面向存量前端系统的 AI Coding 上下文治理 workflow，通过自动扫描源码、抽取项目知识、生成模型上下文资料、提供任务引导 Skill 和轻量质量回检能力，使存量系统能够低成本、可控、可复制地接入 Qwen2.5-Coder-32B。

3. 项目目标

3.1 总体目标

建设一套可复制的“存量前端系统 AI Coding 上下文治理 workflow”，将旧系统中的页面、组件、接口、路由、权限、模板、代码规范等信息转化为 Qwen2.5-Coder-32B 可检索、可拼接、可约束的上下文资料包。

该 workflow 不是传统意义上的文档补全工具，而是面向 Qwen2.5-Coder-32B 的模型上下文治理工具。

3.2 具体目标

1. 自动识别存量前端系统的技术栈、工程结构、构建命令和代码规范。
 2. 自动生成页面清单、组件清单、接口清单、路由权限说明等基础上下文资料。
 3. 抽取典型页面、组件、接口、权限、字典等代码模板。
 4. 生成面向 Qwen2.5-Coder-32B 的短上下文、强约束 Prompt 资料。
 5. 提供专用任务引导 Skill，降低开发人员学习复杂提示词模板的成本。
 6. 提供变更文件 ESLint 轻量质量回检能力，避免旧系统全量 build 压力。
 7. 输出接入与使用指导手册，指导开发人员规范使用 AI Coding。
 8. 支持后续扩展为多系统批量接入、自动生成报告、沉淀优秀案例。
-

4. 非目标范围

第一阶段不解决以下问题：

1. 不默认执行全量 build。
 2. 不默认执行全量 typecheck。
 3. 不默认扫描和治理全部历史代码问题。
 4. 不承诺模型自动完成复杂业务开发。
 5. 不支持大规模架构重构的自动化生成。
 6. 不替代人工 Code Review、业务确认和安全评审。
 7. 不处理后端接口设计和数据库设计。
 8. 不强制统一所有存量系统的工程结构。
-

5. 用户角色

5.1 系统负责人

关注系统是否具备 AI Coding 接入条件，是否完成上下文治理，是否可以推广给普通开发人员使用。

5.2 前端开发人员

关注如何使用模型新增页面、修改页面、接入接口、修复报错，以及如何降低提示词编写成本。

5.3 前端技术小组

关注方案是否可复制、可标准化、可推广，是否能够沉淀为通用 workflow、通用 Skill 和技术规范。

5.4 项目管理与质量人员

关注模型生成代码是否可控、是否可检查、是否有报告、是否避免引入质量风险。

6. 核心设计原则

6.1 面向 Qwen2.5-Coder-32B 优化

本方案的最终服务对象是 Qwen2.5-Coder-32B。考虑该类 32B 代码模型具备较好的成本与性能平衡，但在长上下文理解、复杂项目全局推理、多文件稳定生成方面仍存在限制，因此上下文资料必须遵循以下原则：

1. 短上下文。
2. 强结构。
3. 强约束。
4. 高相关样例。
5. 明确禁止事项。
6. 明确输出格式。
7. 明确轻量质量回检规则。

6.2 不让模型自由发挥

模型不得脱离项目上下文生成代码。所有代码生成任务必须基于：

1. 当前系统 AI Coding 规则。
2. 当前系统典型代码模板。
3. 相似页面示例。
4. 已有组件清单。
5. 已有接口清单。
6. 项目已有 service、request、route、permission、style 规则。

6.3 文档不是给人看的说明书，而是给模型用的上下文资产

工作流生成的文档应避免长篇背景说明，优先采用规则化、结构化、可检索、可拼接进 Prompt 的方式组织。

6.4 增量治理，不做一次性全量清算

存量系统历史包袱较多，第一阶段不追求全量治理所有旧问题，而是优先保障 AI 本次生成和修改的代码可控、可检查、可回修。

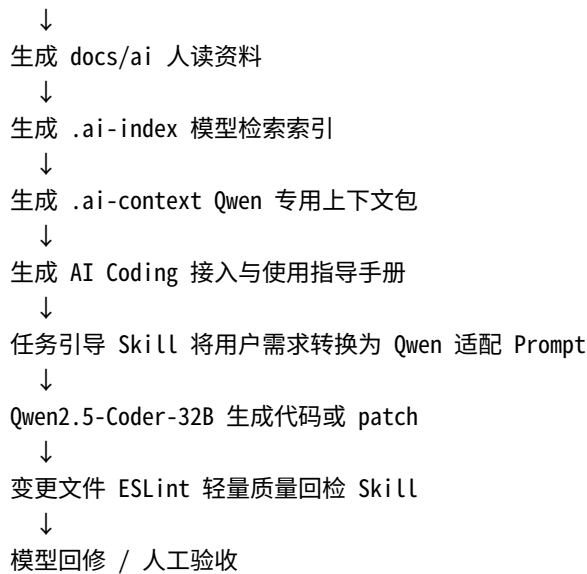
6.5 轻量质量回检优先

默认采用变更文件 ESLint 检查，不默认执行全量 build，避免旧系统因构建耗时、依赖环境复杂、历史问题过多导致接入成本过高。

7. 总体方案

7.1 总体流程

```
graph TD
    A[上传存量前端系统源码] --> B[存量系统上下文治理 Agent]
    B --> C[调用多个专项 Skill 扫描项目]
```



7.2 实现模式

采用：

Agent 编排主流程 + Skill 执行专项能力

其中：

1. Agent 负责整体流程编排、项目类型判断、异常兜底、产物汇总和报告生成。
2. Skill 负责源码解压、技术栈识别、页面扫描、组件扫描、接口扫描、文档生成、Prompt 适配和质量回检等可复用能力。

不建议只用 Agent，也不建议只用 Skill。

8. 功能范围

8.1 存量系统上下文治理 Agent

8.1.1 功能定位

负责对存量前端系统进行整体扫描和治理，生成面向 Qwen2.5-Coder-32B 使用的上下文资料包。

8.1.2 主要职责

1. 接收源码压缩包。
2. 调用源码解压 Skill。
3. 识别项目技术栈。
4. 判断系统类型。
5. 调用页面、组件、接口、路由等扫描 Skill。
6. 组织生成 docs/ai、.ai-index、.ai-context。
7. 汇总异常、缺失项和待人工确认事项。

8. 生成治理报告。
9. 生成接入与使用指导手册。

8.2 项目基础扫描 Skill

8.2.1 输入

1. package.json。
2. lock 文件。
3. 配置文件。
4. src 目录。
5. README 或已有文档。

8.2.2 识别内容

1. 技术栈：Vue2、Vue3、React、Umi 等。
2. 构建工具：Webpack、Vite、Umi、Vue CLI 等。
3. UI 框架：Ant Design、Element UI、Vant、自研组件等。
4. 状态管理：Vuex、Pinia、Redux、model、hooks 等。
5. 请求方式：axios、request、umi-request、fetch 等。
6. 路由方式：静态路由、动态路由、后端菜单等。
7. 样式方案：less、scss、css module、unocss 等。
8. 代码规范：ESLint、Prettier、tsconfig 等。
9. 工程命令：dev、build、lint、test 等。

8.2.3 输出

```
docs/ai/system-profile.md
.ai-index/project-profile.json
```

8.3 页面与路由扫描 Skill

8.3.1 输入

1. 页面目录。
2. 路由配置。
3. 菜单配置。
4. 页面文件。

8.3.2 识别内容

1. 页面名称。
2. 页面路径。
3. 路由路径。
4. 页面类型。
5. 使用组件。
6. 调用接口。
7. 关联权限。

8. 页面标题或菜单名称。

8.3.3 页面类型分类

1. 列表页。
2. 详情页。
3. 新增页。
4. 编辑页。
5. 表单页。
6. 弹窗页。
7. 流程页。
8. 工作台页。
9. 图表页。
10. 嵌套页面。

8.3.4 输出

```
docs/ai/page-inventory.md  
.ai-index/pages.json
```

8.4 组件扫描 Skill

8.4.1 输入

1. components 目录。
2. common components 目录。
3. biz-components 目录。
4. 页面引用关系。

8.4.2 识别内容

1. 组件名称。
2. 文件路径。
3. 适用场景。
4. props。
5. emits 或 callbacks。
6. 引用页面。
7. 使用示例。
8. 是否为公共组件。
9. 是否为业务组件。

8.4.3 输出

```
docs/ai/component-inventory.md  
.ai-index/components.json  
docs/ai/examples/components/
```

8.5 接口扫描 Skill

8.5.1 输入

1. services 目录。
2. api 目录。
3. model/store/hooks。
4. 页面内接口调用。
5. request 封装文件。

8.5.2 识别内容

1. 接口方法名。
2. 文件路径。
3. 请求方式。
4. 请求地址。
5. 入参。
6. 出参。
7. 调用页面。
8. 调用场景。
9. 是否存在类型定义。

8.5.3 输出

```
docs/ai/api-inventory.md
.ai-index/apis.json
```

8.6 页面-操作-接口映射 Skill

8.6.1 功能定位

识别页面中的用户操作、事件处理函数和接口调用之间的关系。

8.6.2 识别内容

1. 页面。
2. 用户操作。
3. 触发方法。
4. 调用接口。
5. 关联组件。
6. 刷新逻辑。
7. 弹窗逻辑。
8. 成功或失败提示。

8.6.3 示例

页面	用户操作	触发方法	调用接口	关联组件
用户列表	点击查询	handleSearch	queryUserList	SearchForm

页面	用户操作	触发方法	调用接口	关联组件
用户列表	点击删除	handleDelete	deleteUser	BizTable
用户编辑	点击保存	handleSubmit	saveUser	UserForm

8.6.4 输出

```
docs/ai/page-action-api-map.md
.ai-index/page-action-api-map.json
```

8.6.5 实施建议

该能力涉及代码语义理解，建议第一版作为增强能力，不作为 MVP 强依赖。

8.7 模板抽取 Skill

8.7.1 目标

从存量系统中抽取典型代码模板，为 Qwen2.5-Coder-32B 生成代码提供高质量参考。

8.7.2 模板类型

1. 列表页模板。
2. 详情页模板。
3. 新增/编辑表单模板。
4. 弹窗表单模板。
5. 字典下拉模板。
6. 表格操作列模板。
7. 权限按钮模板。
8. 接口 service 模板。
9. model/store 模板。
10. 路由配置模板。

8.7.3 模板要求

每个模板应包含：

1. 适用场景。
2. 推荐参考文件。
3. 必须复用组件。
4. 标准代码骨架。
5. 命名规则。
6. 禁止事项。
7. 待确认事项。

8.7.4 输出

```
docs/ai/templates/  
.ai-index/templates.json
```

8.8 AI Coding 规则生成 Skill

8.8.1 目标

生成当前项目专用的模型编码规则，约束 Qwen2.5-Coder-32B 的代码生成行为。

8.8.2 规则内容

1. 项目技术栈。
2. 允许使用的组件。
3. 禁止使用的组件。
4. 接口调用规则。
5. 状态管理规则。
6. 路由规则。
7. 权限规则。
8. 字典规则。
9. 样式规则。
10. 命名规则。
11. 异常处理规则。
12. 输出格式。
13. 质量回检方式。

8.8.3 核心约束

1. 优先复用项目已有组件。
2. 不得新增第三方依赖。
3. 不得编造项目中不存在的组件。
4. 不得编造接口方法。
5. 不得绕过项目 request 封装。
6. 无法确认的字段必须标记 TODO。
7. 修改已有文件时优先输出 patch。
8. 不得修改无关文件。
9. 生成代码后默认执行变更文件 ESLint 轻量检查。

8.8.4 输出

```
docs/ai/ai-coding-rules.md  
.ai-index/rules.json
```

9. 面向 Qwen2.5-Coder-32B 的上下文资料设计

9.1 产物目录

工作流输出三类资料：

docs/ai/	面向研发人员阅读和确认
.ai-index/	面向模型检索和 Prompt 拼接
.ai-context/	面向 Qwen2.5-Coder-32B 的固定上下文策略

9.2 docs/ai 目录

```
docs/  
  ai/  
    README_AI.md  
    system-profile.md  
    page-inventory.md  
    component-inventory.md  
    api-inventory.md  
    page-action-api-map.md  
    route-permission-map.md  
    data-model-inventory.md  
    ai-coding-rules.md  
    AI_CODING_USER_GUIDE.md  
    templates/  
      list-page.template.md  
      detail-page.template.md  
      form-page.template.md  
      modal-form.template.md  
      service.template.md  
      route.template.md  
      permission-button.template.md  
    examples/  
      pages/  
        components/  
        services/
```

9.3 .ai-index 目录

```
.ai-index/  
  project-profile.json  
  pages.json  
  components.json  
  apis.json  
  routes.json  
  page-action-api-map.json  
  templates.json  
  examples.json
```

```
rules.json
quality-check.json
```

9.4 .ai-context 目录

```
.ai-context/
  qwen32b-system-prompt.md
  qwen32b-context-policy.md
  qwen32b-output-format.md
  qwen32b-quality-check.md
```

9.5 Qwen 上下文选择策略

每次任务建议最多注入：

1. 1 份 AI Coding 规则。
2. 1 份任务模板。
3. 1 至 2 个相似页面。
4. 1 至 3 个相关组件示例。
5. 1 至 2 个相关 service 示例。
6. 1 份质量检查规则。

禁止一次性注入全量项目文档或全量代码。

9.6 Qwen 输出格式要求

模型输出必须包含：

1. 变更摘要。
2. 新增文件。
3. 修改文件。
4. unified diff 或 patch。
5. 待确认事项。
6. 轻量质量检查建议。

禁止输出无关解释，不得输出未被要求的大范围重构方案。

10. 专用任务引导 Skill

10.1 Skill 定位

建设一个“前端 AI Coding 任务引导 Skill”，作为使用者与 Qwen2.5-Coder-32B 之间的桥梁。

该 Skill 不直接生成代码，而是负责将开发人员的自然语言需求转换为适合 Qwen2.5-Coder-32B 执行的短上下文、强约束、结构化代码生成任务。

10.2 主要目标

1. 降低开发人员学习复杂 Prompt 模板的成本。
2. 自动识别任务类型。
3. 自动读取 docs/ai、.ai-index、.ai-context。
4. 自动匹配模板和相似代码。
5. 自动生成标准模型请求。
6. 自动生成使用者确认摘要。
7. 自动生成验收 Checklist。

10.3 支持任务类型

MVP 阶段支持以下 4 类：

1. 新增列表页。
2. 修改已有页面。
3. 接入接口。
4. 修复报错。

后续扩展支持：

1. 新增详情页。
2. 新增表单页。
3. 新增组件。
4. 生成测试用例。
5. 补充文档。
6. 解释代码。
7. 生成页面-操作-接口映射。

10.4 输入方式

支持自然语言输入和结构化输入。

自然语言示例：

帮我在客户管理模块新增一个客户列表页，支持客户名称、客户类型、状态查询，表格展示客户编号、客户名称、客户类型、状态、创建时间，支持查看详情和导出。

结构化输入示例：

```
{
  "taskType": "create_list_page",
  "moduleName": "客户管理",
  "pageName": "客户列表",
  "route": "/customer/list",
  "fields": {
    "query": ["客户名称", "客户类型", "状态"],
    "table": ["客户编号", "客户名称", "客户类型", "状态", "创建时间"]
  },
  "actions": ["查询", "重置", "查看详情", "导出"],
}
```

```
"referencePage": "src/pages/user/list/index.tsx",
"api": {
  "list": "queryCustomerList",
  "export": "exportCustomerList"
}
}
```

10.5 输出内容

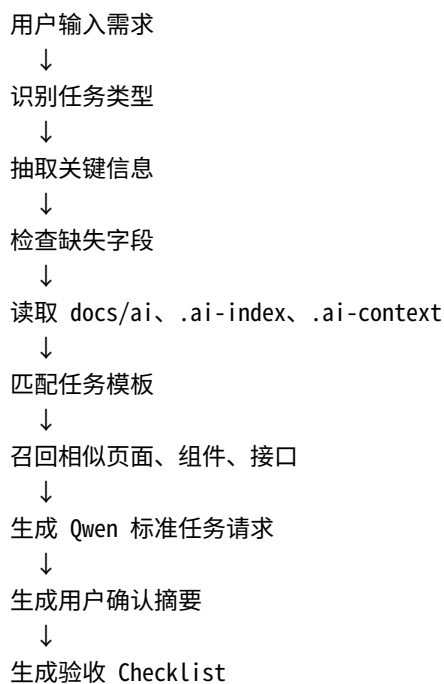
任务引导 Skill 输出两部分：

1. 给 Qwen2.5-Coder-32B 的标准任务请求。
2. 给使用者看的确认摘要和验收清单。

10.6 标准模型请求包含内容

1. 任务类型。
2. 目标模块。
3. 目标页面。
4. 推荐参考页面。
5. 推荐模板。
6. 推荐组件。
7. 推荐接口。
8. 代码生成约束。
9. 输出格式要求。
10. 质量回检要求。
11. 待确认事项。

10.7 Skill 内部流程



11. 变更文件 ESLint 轻量质量回检 Skill

11.1 背景

存量系统普遍存在构建耗时长、依赖环境复杂、历史代码问题多等情况。如果默认执行全量 build 或全量 lint，容易导致接入成本过高，甚至因历史问题影响本次模型生成结果判断。

因此，质量回检阶段第一版采用轻量化增量检查策略。

11.2 定位

质量回检 Skill 第一版定位为：

变更文件 ESLint 轻量回检 Skill

而不是全量质量检查 Skill。

11.3 核心原则

- 1. 谁变更，检查谁。
- 2. 先轻量，后增强。
- 3. 先增量治理，不做一次性全量清算。
- 4. 默认不执行全量 build。
- 5. 默认不执行全量 typecheck。
- 6. 对核心模块、大范围改造、发版前场景可升级检查等级。

11.4 检查分级

等级	检查范围	适用场景	是否默认
L1 轻量检查	仅 ESLint 检查变更文件	日常 AI 生成代码、小范围页面修改	默认
L2 增强检查	ESLint + 局部类型/依赖检查	涉及 service、model、公共组件	可选
L3 全量检查	全量 lint + typecheck + build	发版前、核心模块、大改造	可选

11.5 L1 检查流程





如存在 error，反馈给模型回修

11.6 变更文件识别

支持以下模式：

```
git diff --name-only --diff-filter=ACMRTUXB
```

检查暂存区：

```
git diff --cached --name-only --diff-filter=ACMRTUXB
```

检查相对目标分支：

```
git diff --name-only origin/sit...HEAD --diff-filter=ACMRTUXB
```

11.7 文件过滤范围

默认检查以下类型：

```
.js  
.jsx  
.ts  
.tsx  
.vue
```

11.8 ESLint 执行方式

```
npx eslint <changed-files> -f json
```

实际 CLI 实现中应通过 Node 脚本处理文件列表，避免空列表、路径空格和 Windows 兼容问题。

11.9 输出报告

```
.ai-report/  
  eslint-changed-files.json  
  eslint-changed-files.md
```

11.10 报告内容

1. 本次变更文件清单。
2. 实际检查文件清单。

- 3. Error 数量。
- 4. Warning 数量。
- 5. 可自动修复问题数量。
- 6. 问题明细。
- 7. 修复建议。
- 8. 是否建议进入人工 review。
- 9. 说明：本报告仅覆盖本次变更文件，不代表全量项目质量结果。

12. AI Coding 接入与使用指导手册

12.1 目标

在通用治理 workflow 结束后，自动生成一份《AI Coding 接入与使用指导手册》，指导系统负责人和开发人员如何接入、如何使用、如何验收、如何处理问题。

12.2 手册文件

docs/ai/AI_CODING_USER_GUIDE.md

12.3 手册内容

- 1. 手册目的。
- 2. 适用范围。
- 3. 接入前检查。
- 4. 模型接入方式。
- 5. 推荐使用场景。
- 6. 不建议直接使用的场景。
- 7. 标准使用流程。
- 8. 推荐提示词模板。
- 9. 输出代码验收标准。
- 10. 常见问题处理。
- 11. 使用注意事项。
- 12. 推荐落地方式。

12.4 使用手册定位

该手册不是给模型看的核心上下文，而是给研发人员看的使用说明，重点降低不同系统、不同团队在接入模型时的理解成本。

13. 工作流产物清单

13.1 必选产物

产物	路径	说明
系统基础画像	docs/ai/system-profile.md	说明技术栈、构建工具、工程命令等

产物	路径	说明
页面清单	docs/ai/page-inventory.md	记录页面、路由、文件路径、页面类型等
组件清单	docs/ai/component-inventory.md	记录组件、使用场景、props、示例
接口清单	docs/ai/api-inventory.md	记录接口方法、请求方式、路径、调用页面
AI 编码规则	docs/ai/ai-coding-rules.md	约束模型生成代码
使用指导手册	docs/ai/ AI_CODING_USER_GUIDE.md	指导用户接入和使用
模型索引	.ai-index/*.json	供任务引导 Skill 检索
Qwen 上下文策略	.ai-context/*.md	供 Qwen Prompt 组装使用
轻量检查报告	.ai-report/*.md/json	记录变更文件 ESLint 检查结果

13.2 可选增强产物

产物	路径	说明
页面-操作-接口映射	docs/ai/page-action-api-map.md	记录页面操作和接口调用关系
路由权限说明	docs/ai/route-permission-map.md	记录路由与权限关系
数据模型说明	docs/ai/data-model-inventory.md	记录类型、表单模型、接口模型
典型模板库	docs/ai/templates/	记录列表页、详情页、表单页等模板
示例库	docs/ai/examples/	记录组件、页面、service 示例

14. MVP 范围

14.1 MVP 目标

先完成一套可运行、可复制、能覆盖高频开发场景的最小版本。

14.2 MVP 必须支持

1. 上传或指定存量前端系统源码。
2. 自动识别项目基础信息。
3. 生成 system-profile.md。
4. 生成 page-inventory.md。
5. 生成 component-inventory.md。
6. 生成 api-inventory.md。
7. 生成 ai-coding-rules.md。
8. 生成 .ai-index 基础索引。
9. 生成 .ai-context Qwen 专用上下文。
10. 提供任务引导 Skill。
11. 支持新增列表页、修改已有页面、接入接口、修复报错 4 类任务。

12. 支持变更文件 ESLint 轻量质量检查。
13. 生成 AI_CODING_USER_GUIDE.md。

14.3 MVP 暂不强制支持

1. 页面-操作-接口映射全量准确识别。
 2. 全量 build。
 3. 全量 typecheck。
 4. 自动生成单元测试。
 5. 自动重构复杂模块。
 6. 自动处理跨系统依赖。
 7. 多系统批量治理。
-

15. 后续版本规划

15.1 V1.0：单系统 MVP

完成基础扫描、基础文档生成、Qwen 上下文生成、任务引导 Skill、变更文件 ESLint 检查。

15.2 V1.1：增强上下文治理

增加页面-操作-接口映射、权限规则识别、字典规则识别、模板抽取增强。

15.3 V1.2：多系统推广

支持多个中心、多个系统批量执行，生成系统接入成熟度报告。

15.4 V2.0：闭环代码生成

接入代码生成 Agent，实现需求输入、Prompt 适配、Qwen 生成、ESLint 回检、模型回修、报告输出的完整闭环。

15.5 V2.1：案例沉淀与评测集

沉淀优秀生成案例，构建任务样例库和自动评测集，为后续模型调优和提示词优化提供依据。

16. 成功指标

16.1 接入效率指标

1. 单个旧系统完成基础上下文治理的时间。
2. 文档自动生成覆盖率。
3. 人工确认和修正耗时。
4. 首次接入 Qwen 的准备时间。

16.2 模型生成质量指标

1. 生成代码一次通过 ESLint 的比例。

2. 生成代码中不存在组件幻觉的比例。
3. 生成代码中不存在接口幻觉的比例。
4. 生成代码需要人工大幅调整的比例。
5. 生成 patch 修改范围符合预期的比例。

16.3 使用体验指标

1. 开发人员是否能不学习复杂 Prompt 直接使用。
2. 任务引导 Skill 生成的标准请求是否完整。
3. 使用指导手册是否能支撑普通开发人员独立接入。
4. 常见问题是否能通过手册和 Skill 引导解决。

16.4 质量回检指标

1. 变更文件识别准确率。
 2. ESLint 检查执行成功率。
 3. 轻量报告生成成功率。
 4. 模型根据 ESLint 报错回修成功率。
-

17. 风险与应对

17.1 风险：旧系统结构不规范，扫描结果不完整

应对：

1. 输出待确认事项。
2. 保留人工补充入口。
3. 支持配置目录别名。
4. 支持系统负责人修正索引。

17.2 风险：模型仍然编造组件或接口

应对：

1. 强制注入 ai-coding-rules。
2. 任务引导 Skill 只召回已存在组件和接口。
3. 输出中必须标记 TODO，不得猜测。
4. 质量回检和人工 review 兜底。

17.3 风险：全量 build 压力影响推广

应对：

1. 默认使用变更文件 ESLint 轻量检查。
2. build 作为 L3 可选增强项。
3. 发版前再使用系统原有质量流程。

17.4 风险：开发人员不会写 Prompt

应对：

1. 提供任务引导 Skill。
2. 使用自然语言输入。
3. 自动转换为标准 Qwen 请求。
4. 提供使用指导手册和示例。

17.5 风险：上下文过长导致 Qwen 输出不稳定

应对：

1. 通过 .ai-index 精准召回。
2. 每次任务仅注入最小必要上下文。
3. 控制相似页面、组件、接口示例数量。
4. 不注入全量文档和全量代码。

18. 典型使用场景

18.1 新增列表页

用户输入：

帮我新增一个客户列表页，支持客户名称、客户类型、状态查询，表格展示客户编号、客户名称、客户类型、状态、创建时间，支持查看详情和导出。

流程：

```
任务引导 Skill 识别为新增列表页
↓
召回 list-page.template.md
↓
召回相似列表页
↓
召回 SearchForm、BizTable、AuthButton 示例
↓
召回 customer service 或相似 service 写法
↓
组装 Qwen 标准任务请求
↓
Qwen 输出 patch
↓
ESLint 检查变更文件
↓
生成轻量报告
```

18.2 修改已有页面

用户输入：

帮我在用户列表页增加一个用户状态查询条件，并在表格中增加状态展示。

流程：

任务引导 Skill 识别为修改已有页面
↓
读取目标页面
↓
召回字典组件和表格列示例
↓
组装 Qwen 修改类任务请求
↓
Qwen 输出 patch
↓
ESLint 检查变更文件

18.3 接入接口

用户输入：

帮我把客户列表页查询接口接成 queryCustomerList，接口文件放在 src/services/customer.ts。

流程：

任务引导 Skill 识别为接入接口
↓
召回 service.template.md
↓
召回已有 service 写法
↓
组装 Qwen 任务
↓
Qwen 修改 service 和页面调用
↓
ESLint 检查变更文件

18.4 修复报错

用户输入：

以下是 ESLint 报错，请帮我修复。

流程：

```
任务引导 Skill 识别为修复报错
↓
解析报错文件、行号、规则
↓
只注入相关文件片段
↓
Qwen 生成最小修复 patch
↓
再次执行 ESLint
```

19. 技术实现建议

19.1 CLI 能力

建议提供本地 Node CLI 作为 Skill 的底层执行能力。

示例命令：

```
npx frontend-ai-context init
```

生成 docs/ai、.ai-index、.ai-context。

```
npx frontend-ai-context check --base origin/sit
```

执行变更文件 ESLint 检查。

```
npx frontend-ai-context guide
```

生成或更新使用指导手册。

19.2 Skill 与 CLI 的关系

1. CLI 负责确定性扫描和检查。
2. Skill 负责调用 CLI、组织输入输出。
3. Agent 负责整体编排。
4. Qwen 负责代码生成和修复。

19.3 推荐仓库结构

```
frontend-ai-context-governance/  
  README.md  
  package.json  
  src/  
    cli/  
    scanners/  
      projectScanner.ts  
      pageScanner.ts  
      componentScanner.ts  
      apiScanner.ts  
      routeScanner.ts  
    generators/  
      docsGenerator.ts  
      indexGenerator.ts  
      qwenContextGenerator.ts  
      guideGenerator.ts  
    checks/  
      eslintChangedFilesCheck.ts  
    skills/  
      promptAdapterSkill/  
      qualityCheckSkill/  
  templates/  
    docs/  
    prompts/  
    qwen/  
  examples/
```

20. 最终交付物

20.1 workflow能力

1. 存量系统上下文治理 Agent。
2. 项目基础扫描 Skill。
3. 页面与路由扫描 Skill。
4. 组件扫描 Skill。
5. 接口扫描 Skill。
6. AI Coding 规则生成 Skill。
7. 任务引导 Skill。
8. 变更文件 ESLint 轻量回检 Skill。
9. 使用指导手册生成能力。

20.2 文档产物

1. PRD 文档。
2. AI Coding 接入与使用指导手册。
3. Qwen2.5-Coder-32B 上下文策略说明。
4. 轻量质量回检说明。

5. 示例任务说明。

20.3 系统产物

1. docs/ai 目录。
 2. .ai-index 目录。
 3. .ai-context 目录。
 4. .ai-report 目录。
-

21. 汇报口径

本项目拟建设一套面向 Qwen2.5-Coder-32B 的存量前端系统 AI Coding 上下文治理工作流。该工作流通过 Agent 编排和 Skill 执行的方式，对旧系统源码进行扫描分析，自动生成页面清单、组件清单、接口清单、代码模板、AI 编码规则和模型检索索引，为 Qwen2.5-Coder-32B 提供短上下文、强约束、高相关样例的代码生成条件。

同时，方案提供专用任务引导 Skill，作为开发人员与模型之间的桥梁，将自然语言需求转换为标准化模型任务请求，降低开发人员学习复杂提示词模板的成本。质量回检环节采用变更文件 ESLint 轻量检查，避免旧系统全量 build 压力，实现低成本、可控、可复制的存量系统 AI Coding 接入路径。

该方案的价值不在于让模型直接替代开发，而在于通过上下文治理、任务引导和轻量回检，将模型能力纳入受控研发流程，提升旧系统开发、维护、文档补充和规范治理效率。